SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	Nithimaran D
Register Number	1914260070
Course Code & Name	CSA0211
Slot	D
Assignment / Task Title	**Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative**
GitHub Repository URL	https://github.com/nithimarand883-retro/Assingnment.git
Submission Date	27.10.2026

TASK 1 – Understand the Problem and Identify the Data

1. Aim

To analyse the requirements of the Grama Jyothi Rural Electricity Co-operative billing system, identify the input and output data, select suitable constants, and establish rules for calculating accurate electricity bills.

2. Concept / Theory

The Grama Jyothi Rural Electricity Co-operative provides electricity to approximately 4,000 households. The billing system calculates the electricity consumed by comparing the previous and current meter readings.

The electricity bill consists of two main components: the energy charge and the fixed charge. The energy charge is calculated according to the following tariff slabs:

- First 100 units: Free.
- Units 101–200: Rs. 2.25 per unit.
- Units above 200: Rs. 4.50 per unit.
- Fixed charge: Rs. 50 per household.

The billing program must handle normal meter readings and meter rollover correctly. A 16-bit unsigned meter register stores values from 0 to 65535. When the register exceeds 65535, it rolls over to 0.

To prevent monetary calculation errors, the program should store money in integer paise rather than floating-point rupees.

2.1 Input Data

1. Household Number

Identifies each household uniquely. It helps the co-operative associate the calculated bill with the correct customer.

2. Previous Meter Reading

Stores the electricity meter reading recorded during the previous billing period. It is represented using unsigned short.

3. Current Meter Reading

Stores the latest electricity meter reading. It is also represented using unsigned short.

4. Household Category

Identifies the type of household, such as domestic or commercial. An enum or integer can represent the category.

2.2 Output Data

1. Units Consumed

Represents the electricity consumed during the billing period.

Normal calculation:

Units Consumed = Current Reading − Previous Reading

If the meter rolls over, unsigned arithmetic must be used to calculate the correct consumption.

2. Energy Charge

Represents the cost of electricity consumed according to the tariff slabs. It is stored as an integer number of paise.

3. Fixed Charge

Represents the fixed amount charged to each household. The fixed charge is Rs. 50, equivalent to 5000 paise.

4. Total Bill

Represents the complete amount payable by the household.

Total Bill = Energy Charge + Fixed Charge

2.3 Constants Used

The following values remain unchanged during a single billing run:

- First slab limit: 100 units.
- Second slab limit: 200 units.
- Second slab rate: 225 paise per unit.
- Third slab rate: 450 paise per unit.
- Fixed charge: 5000 paise.

These values should be defined as constants so that tariff revisions can be made in one place.

2.4 Rules for Correct Billing

1. Meter readings must be within the valid range of 0–65535.
2. Electricity consumption must never be negative.
3. The first 100 units must be charged at zero energy cost.
4. Units from 101 to 200 must be charged at 225 paise per unit.
5. Units above 200 must be charged at 450 paise per unit.
6. The total bill must equal the energy charge plus the fixed charge.
7. Monetary calculations must be accurate to the nearest paisa.
8. The household number must identify the correct customer.

2.5 Assumptions

1. The meter uses a 16-bit unsigned register.
2. At most one meter rollover occurs between two readings.
3. Every household pays a fixed charge of Rs. 50.

4. The tariff remains unchanged during one billing run.
5. Each household has valid previous and current meter readings.
6. The basic billing calculation uses the same tariff slabs for all households.

3. Procedure / Algorithm

1. Start the program.
2. Read the household number, previous meter reading, current meter reading, and household category.
3. Validate the meter readings.
4. Calculate the electricity consumption.
5. Handle meter rollover using unsigned arithmetic.
6. Calculate the energy charge according to the tariff slabs.
7. Add the fixed charge to the energy charge.
8. Display the units consumed, energy charge, fixed charge, and total bill.
9. Stop the program.

4. Program / Source Code

The following C program demonstrates the calculation of electricity consumption, including meter rollover.

```
#include <stdio.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
   return (meter_t)(current - previous);
}

int main(void)
{
   printf("Normal consumption: %u units\n",
         consumption(1200, 1530));

   printf("Rollover consumption: %u units\n",
         consumption(65400, 120));
```

```
    return 0;
}
```

5. Output

Normal consumption: 330 units

Rollover consumption: 256 units

Explanation

For normal readings:

Previous Reading = 1200
Current Reading = 1530

Units Consumed = 1530 − 1200 = 330 units.

For rollover readings:

Previous Reading = 65400
Current Reading = 120

The meter has 65536 possible register values, from 0 to 65535.

Units consumed before rollover:

65536 − 65400 = 136 units.

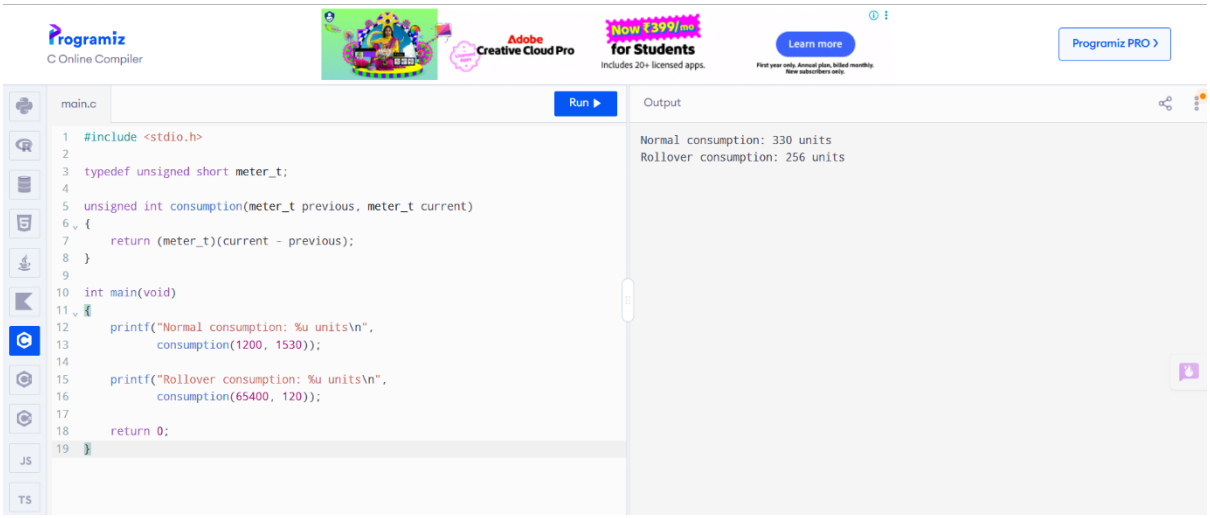
Units consumed after rollover = 120 units.

Therefore:

Total Consumption = 136 + 120 = 256 units.

The result is positive because unsigned arithmetic handles the register rollover.

6. Output Screenshot



7. Alternative Concept and Complexity Analysis

Alternative Concept: Storing Meter Readings as Strings

Instead of storing meter readings as integers, they can be stored as character strings.

For example:

char previous[] = "01200";

char current[] = "01530";

Before calculating consumption, the strings must be converted into integers using a function such as strtoul().

The program can then subtract the converted readings to determine electricity consumption.

Time Complexity

Let d represent the number of digits in a meter reading.

- Converting one string to an integer takes O(d) time.
- Comparing two strings takes O(d) time in the worst case.
- Subtracting fixed-width integer readings takes O(1) time.

For n households, converting both readings for every household requires O(nd) time.

Space Complexity

Representation	Space Required
16-bit integer	Usually 2 bytes per reading
Character string	Approximately d + 1 bytes per reading

The integer representation is more memory-efficient for fixed-width meter registers. Character strings are useful when the original formatting, such as leading zeros, must be preserved.

8. Analysis / Discussion

The analysis identifies the essential inputs, outputs, constants, and validation rules required for a reliable electricity billing system.

Integer meter readings provide efficient arithmetic operations. Unsigned arithmetic handles a single meter rollover correctly, while storing money in integer paise avoids floating-point rounding errors.

The system can be scaled from 4,000 households to 800,000 households by processing each household independently. The total processing time for n households is O(n), assuming a constant amount of work per household.

For smart meters that record readings every 15 minutes, the system would need additional storage for timestamps and interval readings. A time-of-day tariff would also require the billing logic to consider the time at which electricity was consumed.

9. Result

The billing problem was analysed successfully. The required inputs, outputs, constants, assumptions, and validation rules were identified. A C program was implemented to calculate electricity consumption and handle meter rollover correctly.

TASK 2 – CHOOSE THE DATA TYPES

1. Aim

To select suitable C data types for the electricity billing system, examine their sizes and ranges using sizeof, limits.h, and float.h, and demonstrate how unsigned arithmetic and integer-based monetary calculations prevent overflow and precision errors.

2. Concept / Theory

C provides different data types for storing characters, integers, and floating-point values. The appropriate data type must be selected based on the range, precision, and storage requirements of the billing system.

2.1 Selection of Data Types

The following data types are suitable for the Grama Jyothi electricity billing system.

Data Item	C Data Type	Reason
Household number	unsigned int or character array	Identifies each household
Previous meter reading	unsigned short	Stores a 16-bit meter reading
Current meter reading	unsigned short	Stores the latest meter reading
Units consumed	unsigned int	Stores non-negative consumption
Household category	enum	Represents household categories
Energy charge	long	Stores the charge in paise
Fixed charge	long	Stores the fixed amount in paise
Total bill	long or long long	Stores the complete bill amount
Household counter	unsigned int or size_t	Counts processed households
Tariff rates	Integer constants	Maintains consistent tariff values

The actual size and range of each type must be checked on the target system because C implementations can differ.

2.2 Meter Reading and Unsigned Arithmetic

The meter register is defined as:

typedef unsigned short meter_t;

On the target system, this type is expected to occupy 2 bytes and represent values from 0 to 65535.

When the register reaches 65535, the next increment returns it to 0.

The consumption formula is:

$$\text{Consumption} = (\text{Current} - \text{Previous})\ 65536$$

For example:

Previous reading = 65400
Current reading = 120

$$\text{Consumption} = (120 - 65400)\ 65536$$
$$\text{Consumption} = 256 \text{ units}$$

Therefore, unsigned arithmetic correctly handles a single rollover without producing a negative consumption value.

2.3 Why Money Should Be Stored as Integer Paise

Floating-point numbers cannot represent many decimal fractions exactly in binary.

For example, Rs. 112.50 can instead be stored as:

$$112.50 \times 100 = 11250 \text{ paise}$$

Similarly, Rs. 50.00 is stored as 5000 paise.

Integer paise storage is preferred because:

1. Every whole-paisa amount can be represented exactly.
2. Repeated monetary additions do not introduce floating-point rounding errors.
3. Bills can be displayed in rupees and paise using integer division and remainder.
4. The system produces consistent financial results.

However, the integer type must be sufficiently wide to store the maximum expected bill and cumulative total.

3. Procedure / Algorithm

1. Start the program.
2. Include the stdio.h and limits.h header files.
3. Define meter_t as unsigned short.
4. Create a function named consumption().
5. Calculate the difference between the previous and current readings using unsigned arithmetic.
6. Convert the result to the meter register type to handle rollover.
7. Display the size and maximum reading of the meter register.
8. Test normal consumption using readings 1200 and 1530.
9. Test rollover consumption using readings 65400 and 120.
10. Compare the output with the expected output.
11. Stop the program.

4. Program / Source Code

Complete the Code – TODO 1

The following program completes the given code without using if–else.

```
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter:
   after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    return (meter_t)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);

    printf("Consumption: %u units\n",
           consumption(1200, 1530));

    printf("Consumption: %u units\n",
           consumption(65400, 120));

    return 0;
}
```

Explanation of the Completed Code

The expression:

```
return (meter_t)(current - previous);
```

calculates the difference between the two meter readings and converts the result back to the unsigned meter register type.

For a 16-bit unsigned type, conversion produces the result modulo 65536.

This means that if the current reading is smaller than the previous reading because of a rollover, the calculation still produces the correct consumption for one rollover.

Important assumption: The code requires unsigned short to be 16 bits and assumes no more than one rollover occurs between readings. The program should validate readings and use a wider return type for consumption when supporting multiple rollover possibilities or different meter configurations.

5. Output

Expected Output

sizeof(meter_t) = 2 bytes, maximum reading = 65535

Consumption: 330 units

Consumption: 256 units

Explanation of the Output

Case 1: Normal Consumption

Previous reading = 1200
Current reading = 1530

$$1530 - 1200 = 330 \text{ units}$$

Therefore, the output is:

Consumption: 330 units

Case 2: Meter Rollover

Previous reading = 65400
Current reading = 120

Units consumed before rollover:

$$65536 - 65400 = 136$$

Units consumed after rollover:

$$120$$

Total consumption:

$$136 + 120 = 256 \text{ units}$$

Therefore, the output is:

Consumption: 256 units

The result is 256 rather than a negative number because the calculation uses unsigned arithmetic.

6. Data Type Sizes and Ranges

The following table shows typical sizes and ranges on common modern GCC systems. Your actual results must be obtained using the compiler installed on your computer.

Data Type	Typical Size	Typical Range
char	1 byte	−128 to 127, or 0 to 255, depending on implementation
short	2 bytes	−32768 to 32767
unsigned short	2 bytes	0 to 65535
int	4 bytes	−2147483648 to 2147483647
unsigned int	4 bytes	0 to 4294967295
long	4 or 8 bytes	Depends on the system
long long	8 bytes	Typically −9223372036854775808 to 9223372036854775807
float	4 bytes	Approximately $\pm 3.4 \times 10^{38}$
double	8 bytes	Approximately $\pm 1.7 \times 10^{308}$

Note: The ranges shown are typical for the stated sizes. The C standard defines minimum requirements rather than guaranteeing every size in this table.

Program to Check Sizes and Ranges

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
   printf("CHAR_BIT = %d\n", CHAR_BIT);

   printf("sizeof(char) = %zu bytes\n", sizeof(char));
   printf("sizeof(short) = %zu bytes\n", sizeof(short));
   printf("sizeof(unsigned short) = %zu bytes\n",
         sizeof(unsigned short));
   printf("sizeof(int) = %zu bytes\n", sizeof(int));
   printf("sizeof(unsigned int) = %zu bytes\n",
         sizeof(unsigned int));
   printf("sizeof(long) = %zu bytes\n", sizeof(long));
```

```
    printf("sizeof(long long) = %zu bytes\n",
           sizeof(long long));
    printf("sizeof(float) = %zu bytes\n", sizeof(float));
    printf("sizeof(double) = %zu bytes\n", sizeof(double));

    printf("\nInteger Limits:\n");
    printf("USHRT_MAX = %u\n", (unsigned)USHRT_MAX);
    printf("INT_MAX = %d\n", INT_MAX);
    printf("UINT_MAX = %u\n", UINT_MAX);
    printf("LONG_MAX = %ld\n", LONG_MAX);
    printf("LLONG_MAX = %lld\n", LLONG_MAX);

    printf("\nFloating-Point Limits:\n");
    printf("FLT_MAX = %e\n", FLT_MAX);
    printf("DBL_MAX = %e\n", DBL_MAX);
    printf("FLT_EPSILON = %e\n", FLT_EPSILON);
    printf("DBL_EPSILON = %e\n", DBL_EPSILON);

    return 0;
}
```

Compilation and Execution

Compile the program:

```
gcc -std=c11 -Wall -Wextra datatype_sizes.c -o datatype_sizes
```

Run it on Windows:

```
.\datatype_sizes.exe
```

Run it on Linux:

```
./datatype_sizes
```

The output displays the actual sizes, integer limits, and floating-point limits supported by your compiler.

7. Floating-Point Precision Example

Consider adding 4,000 electricity bills, each worth Rs. 0.10.

The exact mathematical total is:

$$4000 \times 0.10 = \text{Rs. } 400.00$$

However, 0.10f cannot be represented exactly as a binary floating-point value. Repeated addition may therefore produce a total slightly different from Rs. 400.00.

Program

```
#include <stdio.h>


int main(void)
{
   float total = 0.0f;
   long total_paise = 0;

   for (int i = 0; i < 4000; i++)
   {
      total += 0.10f;
      total_paise += 10;
   }

   printf("Float total: %.9f\n", total);

   printf("Integer total: Rs %ld.%02ld\n",
         total_paise / 100,
         total_paise % 100);

   return 0;
}
```

Output

On a typical GCC system using IEEE 754 single-precision floating-point arithmetic, the output is:

Float total: 400.015197754

Integer total: Rs 400.00

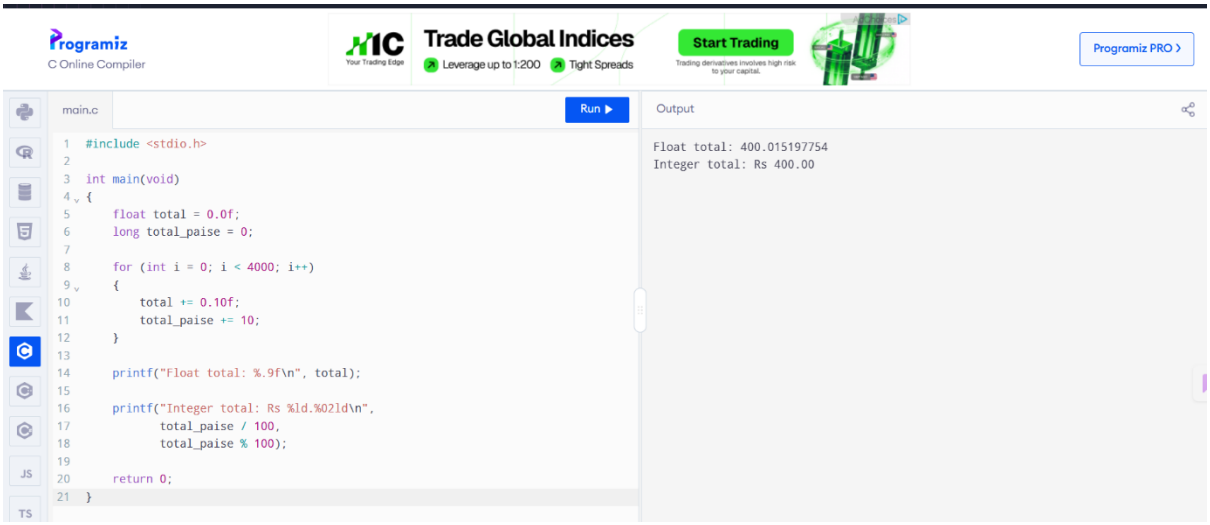
The precise floating-point output depends on the implementation and compiler settings.

The integer total is exactly Rs. 400.00 because each bill is represented as 10 paise and the program adds 10 paise exactly 4,000 times.

Therefore, integer paise is the more reliable representation for this billing system.

8. Output Screenshot

Capture screenshots of the following program outputs:



9. Alternative Concept and Complexity Analysis

Alternative Concept: Storing Money as double Rupees

Instead of storing money as integer paise, the program could store monetary values as double.

For example:

double fixed_charge = 50.00;

double energy_charge = 112.50;

double total_bill = fixed_charge + energy_charge;

The program would calculate the energy charge using floating-point arithmetic and round the final bill to the nearest paisa when displaying or storing the payable amount.

For example, the final amount could be rounded to two decimal places for display. However, formatting with %.2f alone only changes the displayed representation; it does not change the stored value.

For a reliable financial system, the rounding rule must be explicitly defined.

Time Complexity Comparison

Operation	**Integer Paise**	**Double Rupees**
Addition	O(1)	O(1)
Multiplication	O(1)	O(1)
Total for n bills	O(n)	O(n)
Currency precision	Exact for whole-paisa amounts	Approximate binary representation

Space Complexity Comparison

A double typically occupies 8 bytes. A long occupies either 4 or 8 bytes, depending on the system.

Both representations require constant space per value, or O(1) space.

Rounding Error

When double values are added repeatedly, small floating-point errors can accumulate. The final error depends on the values, order of operations, and floating-point implementation.

Integer paise avoids such representation errors for amounts expressed in whole paise, provided that the integer type does not overflow.

Therefore, integer paise is preferred for electricity billing.

10. Analysis / Discussion

The program demonstrates the importance of selecting appropriate data types for a financial application.

The unsigned short type is suitable for the meter register when it supports the range 0–65535. Unsigned arithmetic correctly handles a single rollover and produces 256 units for the readings 65400 and 120.

The limits.h and float.h headers help verify the actual limits of the selected data types. The floating-point comparison shows why storing monetary values in integer paise prevents errors when thousands of bills are processed.

For district-level billing, the program should use a sufficiently wide integer type for cumulative amounts and validate all readings before calculating bills.

11. Result

Suitable C data types were selected and analysed using sizeof, limits.h, and float.h. The meter-consumption program correctly calculated 330 units for normal readings and 256 units for rollover readings. The floating-point comparison demonstrated that integer paise provides more reliable monetary calculations for the electricity billing system.

TASK 3 – DEFINE THE CONSTANTS

1. Aim

To define electricity tariff values, slab limits, and fixed charges using constants in C, and implement a billing program that calculates the correct electricity bill according to the specified tariff slabs.

2. Concept / Theory

Constants are fixed values that do not change during a program's execution. In the Grama Jyothi Rural Electricity Co-operative billing system, tariff rates, slab limits, and fixed charges should be defined as constants to ensure consistency and simplify tariff revisions.

C provides three common ways to represent constants: #define, const, and enum.

2.1 Using #define

The #define directive defines a symbolic name that the preprocessor replaces with its specified value before compilation.

Example:

#define RATE2_PAISE 225

Here, RATE2_PAISE represents 225 paise per unit.

Advantages:

- Tariff values can be maintained in one place.
- Revisions can be made easily.
- The preprocessor replaces the macro name before compilation.
- No storage is allocated for the macro itself.

2.2 Using const

The const keyword declares an object whose value cannot be modified through that identifier after initialization.

Example:

```
const long fixed_charge = 5000L;
```

This represents a fixed charge of 5000 paise.

Advantages:

- Provides type information and normal type checking.
- Constants can be inspected in a debugger.
- Helps prevent accidental modification.

A const object is not the same as a preprocessor constant and cannot be used in every context that requires a compile-time integer constant expression in C.

2.3 Using enum

The enum keyword defines named integer constants.

Example:

```
enum HouseholdCategory
{
   DOMESTIC = 1,
   COMMERCIAL = 2
};
```

This is useful for representing household categories or program status values.

2.4 Tariff Constants

The following constants are used in the electricity billing system.

Constant	Value	Description
SLAB1_UNITS	100	First slab limit
SLAB2_UNITS	200	Second slab limit

Constant	Value	Description
RATE1_PAISE	0	Rate for the first slab
RATE2_PAISE	225	Rate for the second slab
RATE3_PAISE	450	Rate for the third slab
FIXED_CHARGE_PAISE	5000	Fixed charge of Rs. 50

All monetary values are represented in paise to avoid floating-point rounding errors.

3. Procedure / Algorithm

1. Start the program.
2. Include the stdio.h header file.
3. Define the slab limits, tariff rates, and fixed charge using #define.
4. Create a function named compute_bill() that accepts the number of units consumed.
5. Initialize the bill amount with the fixed charge.
6. If consumption exceeds 200 units, calculate the charge for units above 200 at 450 paise per unit.
7. Calculate the charge for units between 101 and 200 at 225 paise per unit.
8. If consumption is between 101 and 200, calculate only the second-slab charge.
9. Add the energy charge to the fixed charge.
10. Return the total bill in paise.
11. Display the bill in rupees and paise.
12. Stop the program.

4. Program / Source Code

The following program completes TODO 1 using the macros defined at the beginning of the given code.

```
#include <stdio.h>


#define SLAB1_UNITS 100

#define SLAB2_UNITS 200


#define RATE1_PAISE 0

#define RATE2_PAISE 225

#define RATE3_PAISE 450
```

```
#define FIXED_CHARGE_PAISE 5000L

/* Returns the bill in paise. */
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS)
    {
        paise += (long)(units - SLAB2_UNITS)
                * RATE3_PAISE;

        paise += (long)(SLAB2_UNITS - SLAB1_UNITS)
                * RATE2_PAISE;
    }
    else if (units > SLAB1_UNITS)
    {
        paise += (long)(units - SLAB1_UNITS)
                * RATE2_PAISE;
    }

    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];

    for (int i = 0; i < n; i++)
    {
        long p = compute_bill(tests[i]);
```

```
        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }

    return 0;
}
```

Explanation of the Program

The compute_bill() function calculates the total bill in paise.

Initially, the bill is set to 5000 paise, representing the fixed charge of Rs. 50.

If the consumption exceeds 200 units, the program calculates:

- 100 units at 225 paise per unit.
- All units above 200 at 450 paise per unit.

If consumption is between 101 and 200 units, only the units above 100 are charged at 225 paise per unit.

The first 100 units are free, so no energy charge is added for that slab.

Finally, the program returns the total bill in paise.

5. Output

Expected Output

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

Explanation of the Output

Case 1: 0 Units

Fixed charge = Rs. 50.00
Energy charge = Rs. 0.00

Total bill = Rs. 50.00

Case 2: 100 Units

The first 100 units are free.

Total bill = Rs. 50.00

Case 3: 150 Units

Fixed charge = Rs. 50.00

Energy charge:

$$(150 - 100) \times 2.25 = \text{Rs. } 112.50$$

Total bill:

$$50 + 112.50 = \text{Rs. } 162.50$$

Case 4: 200 Units

Fixed charge = Rs. 50.00

Energy charge:

$$100 \times 2.25 = \text{Rs. } 225.00$$

Total bill:

$$50 + 225 = \text{Rs. } 275.00$$

Case 5: 350 Units

Fixed charge = Rs. 50.00

Second slab charge:

$$100 \times 2.25 = \text{Rs. } 225.00$$

Third slab charge:

$$(350 - 200) \times 4.50 = \text{Rs. } 675.00$$

Total bill:

$$50 + 225 + 675 = \text{Rs. } 950.00$$

6. Tariff Revision Test

One advantage of defining tariff values as constants is that the rates can be updated without modifying the billing logic.

Change the following line:

#define RATE2_PAISE 225

to:

#define RATE2_PAISE 230

Compile and execute the program again.

Revised Expected Output

0 units -> Rs 50.00

100 units -> Rs 50.00

150 units -> Rs 165.00

200 units -> Rs 280.00

350 units -> Rs 955.00

Explanation

For 150 units, the second slab contains 50 charged units.

The revised charge is:

$$50 \times 2.30 = \text{Rs. } 115.00$$

Therefore:

$$50 + 115 = \text{Rs. } 165.00$$

For 350 units, the second slab charge increases by Rs. 5.00, so the total becomes Rs. 955.00.

The first 100 units remain free, and the third-slab rate remains unchanged.

Note: The revised output is obtained after recompiling the program with the changed macro.

7. Output Screenshot



8. Alternative Concept and Complexity Analysis

Alternative Concept: Using const Variables Instead of #define

The tariff values could be represented using const variables.

Example:

const int slab1_units = 100;

```
const int slab2_units = 200;


const long rate2_paise = 225L;
const long rate3_paise = 450L;


const long fixed_charge_paise = 5000L;
```

The billing function would use these variables instead of the macros.

This approach provides type information and allows the debugger to inspect the constants. However, a const variable is an object, and it cannot replace a macro in every context requiring a compile-time constant expression.

An enum could represent household categories, but it is not the best choice for monetary tariff values.

Comparison of Constants

Feature	#define	const	enum
Purpose	Text substitution	Read-only object	Named integer constants
Type checking	No type associated with the macro itself	Yes	Integer-based
Scope	Controlled by preprocessing directives and source placement	Normal C scope rules	Normal C scope rules
Debugging	May not appear as an object	Can often be inspected	Enumerator names may be available in debugging information
Storage	Macro itself uses no storage	Object may occupy storage	Enumerator itself uses no object storage
Running time	No inherent runtime cost	Usually no inherent runtime cost	No inherent runtime cost

Time Complexity

The compute_bill() function performs a fixed number of comparisons, multiplications, and additions.

Therefore, the time complexity for calculating one bill is:

$$\boxed{O(1)}$$

For n households, processing every bill takes:

$$\boxed{O(n)}$$

Changing the constants from #define to const does not change the algorithm's asymptotic time complexity.

Space Complexity

The billing function uses a fixed number of variables, so its auxiliary space complexity is:

$$\boxed{O(1)}$$

The complete billing system requires O(n) storage if all n bills are stored simultaneously. If each bill is calculated and processed one at a time, the additional working space can remain O(1), excluding input/output storage.

9. Analysis / Discussion

The program demonstrates how symbolic constants improve the reliability and maintainability of the billing system.

All tariff rates and slab limits are defined in one place. Consequently, a tariff revision can be implemented by modifying the relevant macro and recompiling the program.

The use of integer paise prevents floating-point rounding errors. The program correctly calculates bills for consumption values of 0, 100, 150, 200, and 350 units.

The comparison of #define, const, and enum shows that each has a different purpose. For this assignment, #define is appropriate because the given program specifically requires the supplied macros to be used.

10. Result

The tariff constants were defined successfully using #define, and the compute_bill() function was implemented to calculate electricity charges according to the specified slabs.

TASK 4 – USE MACROS SAFELY

1. Aim

To implement safe C macros for repeated calculations, understand how macros are expanded by the preprocessor, identify common macro errors, and compare macros with static inline functions in terms of type checking, execution time, and code size.

2. Concept / Theory

A macro is defined using the #define preprocessor directive in C. Before the actual compilation process, the preprocessor replaces the macro name with its replacement text.

Macros are useful for small, repeated calculations. However, they must be written carefully to avoid errors involving operator precedence and repeated evaluation of arguments.

2.1 Macro Expansion

Consider the following macro:

```
#define SQUARE(x) ((x) * (x))
```

When the program contains:

SQUARE(5)

The preprocessor expands it to:

((5) * (5))

The expression is then evaluated as:

$$5 \times 5 = 25$$

2.2 Why Parentheses Are Necessary

Consider an incorrectly written macro:

#define SQUARE(x) x * x

If the argument is a + 1, the expression:

SQUARE(a + 1)

expands to:

a + 1 * a + 1

If a = 3, the result becomes:

$$3 + 1 \times 3 + 1 = 7$$

However, the correct square is:

$$(3 + 1)^2 = 16$$

Therefore, the correct macro is:

#define SQUARE(x) ((x) * (x))

Parentheses around the argument and the entire expression prevent operator-precedence errors.

2.3 Safe MAX Macro

The maximum of two values can be calculated using the following macro:

#define MAX(a, b) (((a) > (b)) ? (a) : (b))

This macro compares the two arguments and returns the larger value.

However, it can evaluate an argument more than once.

For example:

MAX(i++, j)

If i++ is used as the first argument, i may be incremented during the comparison and again if that argument is selected.

Therefore, macros should not receive arguments containing side effects, such as i++, when the macro may evaluate them multiple times.

2.4 When Is a Function Better Than a Macro?

A function is generally preferable when:

1. Type checking is important.
2. Arguments must be evaluated only once.
3. The operation contains complex logic.
4. Debugging and maintainability are important.
5. Repeated macro expansion could unnecessarily increase code size.

A static inline function can provide the benefits of a normal function while allowing the compiler to replace the function call with inline instructions when appropriate.

3. Procedure / Algorithm

1. Start the program.
2. Include the stdio.h header file.
3. Define a macro to convert rupees into paise.
4. Define a fully parenthesized SQUARE macro.
5. Define a fully parenthesized MAX macro.
6. Create equivalent static inline functions.
7. Test the macros using sample values.
8. Display the calculated results.
9. Compile the program using GCC.
10. Execute gcc -E -P to examine the expanded source code.
11. Compare the macro approach with the inline-function approach.
12. Stop the program.

4. Program / Source Code

The following C program implements safe macros and equivalent inline functions.

```
#include <stdio.h>


#define RUPEES_TO_PAISE(r) ((r) * 100L)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))


static inline int square_function(int x)
{
   return x * x;
}
```

```
static inline int max_function(int a, int b)
{
    return (a > b) ? a : b;
}

int main(void)
{
    int a = 3;
    int b = 5;

    printf("Square of a + 1: %d\n", SQUARE(a + 1));

    printf("Maximum of a and b: %d\n", MAX(a, b));

    printf("100 rupees in paise: %ld\n",
           RUPEES_TO_PAISE(100));

    printf("Square using inline function: %d\n",
           square_function(a + 1));

    printf("Maximum using inline function: %d\n",
           max_function(a, b));

    return 0;
}
```

Explanation of the Program

1. RUPEES_TO_PAISE

#define RUPEES_TO_PAISE(r) ((r) * 100L)

This macro converts an amount in whole rupees into paise.

For example:

$$100 \times 100 = 10000 \text{ paise}$$

2. SQUARE

#define SQUARE(x) ((x) * (x))

This macro calculates the square of a value while protecting the expression from operator-precedence errors.

3. MAX

#define MAX(a, b) (((a) > (b)) ? (a) : (b))

This macro compares two values and returns the larger one.

4. Inline Functions

The functions square_function() and max_function() perform the same calculations as the corresponding macros.

They provide normal function-parameter type checking and evaluate each argument once when called.

5. Output

Expected Output

Square of a + 1: 16

Maximum of a and b: 5

100 rupees in paise: 10000

Square using inline function: 16

Maximum using inline function: 5

Explanation of the Output

Case 1: Square of a + 1

Given:

a = 3

$$(3 + 1)^2 = 4^2 = 16$$

Therefore, the output is 16.

Case 2: Maximum of a and b

Given:

a = 3 and b = 5

Since 5 is greater than 3, the output is 5.

Case 3: Conversion of Rupees into Paise

$$100 \times 100 = 10000 \text{ paise}$$

Therefore, the output is 10000.

Case 4: Square Using Inline Function

The function calculates:

$$(3 + 1)^2 = 16$$

Therefore, the output is 16.

Case 5: Maximum Using Inline Function

The function compares 3 and 5 and returns 5.

6. Procedure to View Macro Expansion Using GCC

The GCC preprocessor allows us to inspect how macros are expanded before compilation.

Step 1: Save the Program

Save the source code in a file named:

main.c

Step 2: Preprocess the Program

Execute the following command:

gcc -E -P main.c

Here:

- gcc invokes the GCC compiler.
- -E runs only the preprocessor.
- -P suppresses line-marker information in the output.

Step 3: Save the Expanded Code

To save the preprocessed output in a file, execute:

gcc -E -P main.c -o preprocessed.c

Step 4: Examine the Expanded Expression

The original macro:

SQUARE(a + 1)

expands to:

((a + 1) * (a + 1))

The original macro:

MAX(a, b)

expands to:

(((a) > (b)) ? (a) : (b))

This demonstrates how the preprocessor replaces macro calls with their corresponding expressions.

Step 5: Compile the Program

gcc -std=c11 -Wall -Wextra main.c -o main

Step 6: Execute the Program

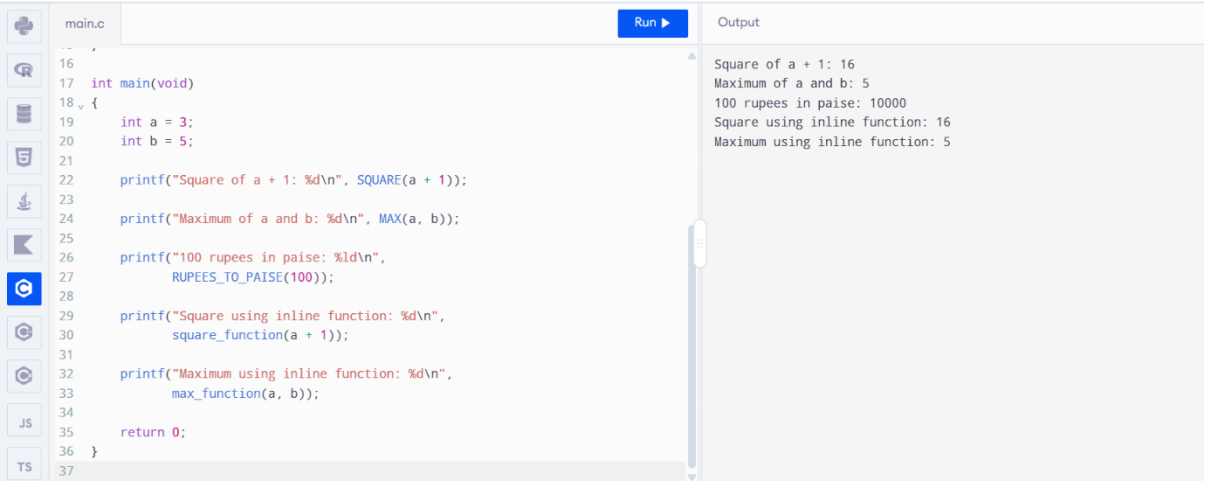
On Windows using GCC:

.\main.exe

On Linux:

./main

7. Output Screenshot



8. Alternative Concept and Complexity Analysis

Alternative Concept: Replacing Macros with Static Inline Functions

Instead of using macros, the same calculations can be implemented using static inline functions.

For example:

```
static inline int square_function(int x)
{
    return x * x;
}

static inline int max_function(int a, int b)
{
    return (a > b) ? a : b;
}
```

The functions calculate the square of an integer and the maximum of two integers.

Unlike a macro, a function evaluates each argument once at the call site. It also provides normal parameter type checking.

However, inline does not guarantee that the compiler will eliminate the function call. The compiler decides whether to inline the function based on its optimization decisions.

Comparison Between Macros and Inline Functions

Feature	Macro	Static Inline Function
Processing	Expanded by the preprocessor	Processed by the compiler
Type checking	No normal function-parameter type checking	Provides parameter type checking
Argument evaluation	May evaluate an argument multiple times	Each argument is evaluated once at the call site
Function-call overhead	No function call is inherently required	May be eliminated by inlining
Code size	Replacement text is expanded at every use	May share code if not inlined
Debugging	Can be difficult	Generally easier
Time complexity	O(1) for simple arithmetic	O(1) for simple arithmetic

Time Complexity

The SQUARE macro performs one multiplication, so its time complexity is:

$$\boxed{O(1)}$$

The MAX macro performs a comparison and selects a value, so its time complexity is:

$$\boxed{O(1)}$$

The equivalent inline functions also have O(1) time complexity for these operations.

If the macro or function is used k times, the total time complexity is:

$$\boxed{O(k)}$$

This assumes that each use performs a constant amount of work.

Space Complexity and Code Size

Macros are expanded at every use. Therefore, when a macro is used k times, its replacement text may be copied into the generated code at each use, increasing code size.

A non-inlined function may use one shared function body. An inline function may also produce repeated machine instructions if the compiler inlines every call.

Thus, the actual code size depends on the compiler and optimization settings.

For simple arithmetic, both approaches require O(1) auxiliary space per operation, excluding generated machine-code storage.

Conclusion of the Comparison

Macros are useful for small repeated expressions, but they must be written carefully to avoid precedence and repeated-evaluation errors.

static inline functions are generally safer when type checking, predictable argument evaluation, and easier debugging are important.

9. Analysis / Discussion

The program demonstrates the correct use of fully parenthesized macros for squaring a number, finding the maximum of two values, and converting rupees into paise.

Parentheses prevent operator-precedence errors. However, they do not prevent repeated argument evaluation in macros such as MAX.

Using gcc -E -P makes it possible to inspect the preprocessor's output and understand how macro definitions are expanded before compilation.

The comparison with inline functions shows that both approaches can perform simple calculations efficiently, but inline functions provide stronger type checking and safer argument evaluation.

10. Result

Safe C macros were implemented and tested successfully. The program correctly calculated the square of a value, found the maximum of two integers, and converted rupees into paise.

The macro expansion was examined using GCC, and the advantages and limitations of macros were compared with those of static inline functions.